

Docker and Kubernetes



Lesson Objectives

Session 1 – Getting started with Docker

Session 2 – Building Docker images

Session 3 – Docker Networking

Session 4 – Docker storage

Session 5 – Making a real project with Docker and NodeJs

Session 6 – Docker Swarm

Session 7-10 – Kubernetes

Session 11 – Final test

Session 8

Kubernetes (Cont.)

Agenda

- Assignment Review & Guides
- Kubernetes concepts(Namespace, Pod, Deployment, Labels/Selector, ReplicaSet)
- Working with Pod
- Lab: Deployment Rolling Update and Rollback

API

- The **REST API** is the true **keystone** of Kubernetes.
- **Everything** within Kubernetes is as an **API Object**.
- Referenced within an object as the **apiVersion** and **kind**.

Format:

/apis/<group>/<version>/<resource>

Examples:

/apis/apps/v1/deployments

/apis/batch/v1beta1/cronjobs

- Objects are a “record of intent” or a persistent entity that represent the desired state of the object within the cluster.
- All objects **MUST** have `apiVersion`, `kind`, and poses the nested fields `metadata.name`, `metadata.namespace`, and `metadata.uid`.

- Files or other representations of Kubernetes Objects are generally represented in YAML.
- Three basic data types:
 - **mappings** – hash or dictionary,
 - **sequences** – array or list
 - **scalars** – string, number, boolean etc

```
apiVersion: v1
kind: Pod
metadata:
  name: sample
  namespace: test
spec:
  containers:
  - name: container1
    image: nginx
  - name: container2
    image: alpine
```


YAML VS JSON

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
spec:
  containers:
  - name: nginx
    image: nginx:stable-alpine
    ports:
    - containerPort: 80
```

Are you wondering about the YAML schema? `kubectl explain` is your friend ;)
What about kinds or versions? `kubectl api-versions` is your friend ;)

```
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "name": "pod-example"
  },
  "spec": {
    "containers": [
      {
        "name": "nginx",
        "image": "nginx:stable-alpine",
        "ports": [
          { "containerPort": 80 }
        ]
      }
    ]
  }
}
```

CORE CONCEPTS

Known as
Projects in
OpenShift

Namespaces are a logical cluster or environment, and are the primary method of partitioning a cluster or scoping access.

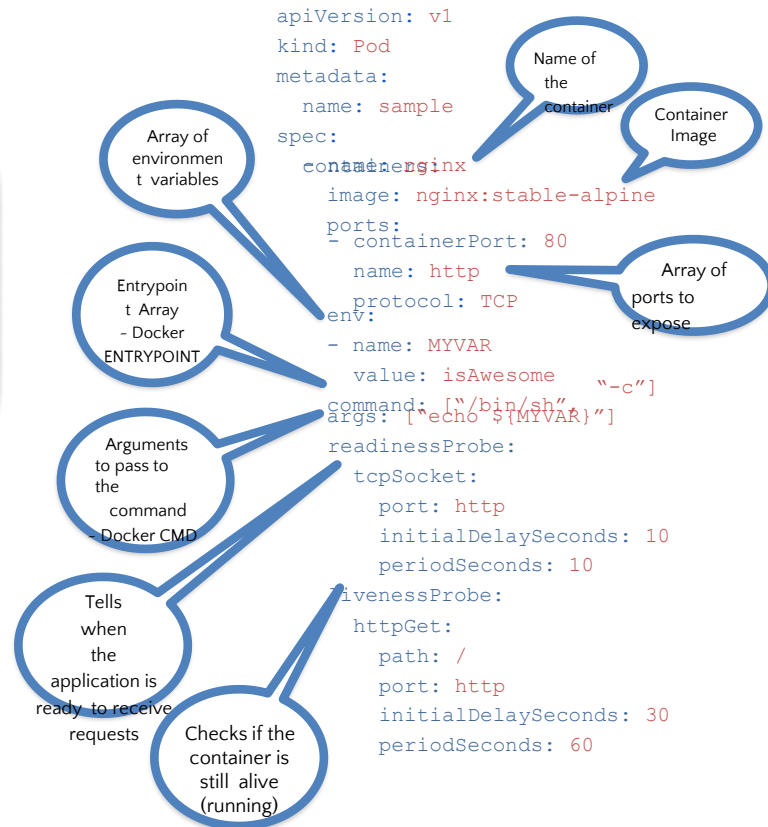
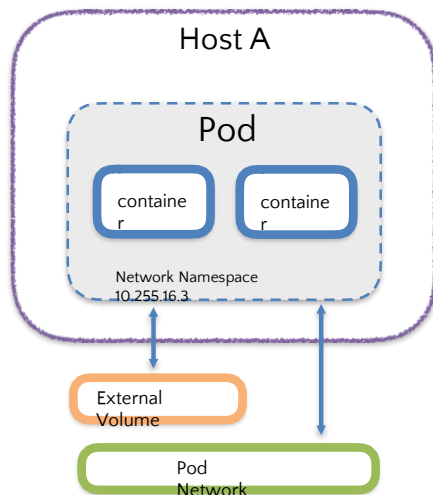
```
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    app: MyBigWebApp
```

```
$ kubectl get ns --show-labels
```

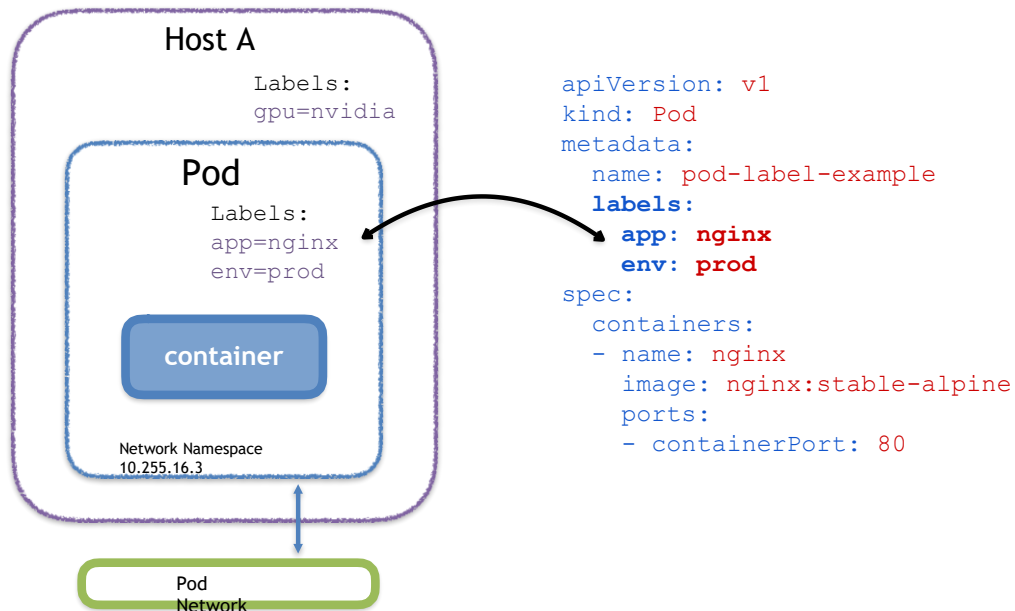
NAME	STATUS	AGE	LABELS
default	Active	11h	<none>
kube-public	Active	11h	<none>
kube-system	Active	11h	<none>
prod	Active	6s	app=MyBigWe

bApp

- **Atomic unit** or smallest “*unit of work*” of Kubernetes.
- Foundational building block of Kubernetes Workloads.
- Pods are one or MORE containers that share volumes, a network namespace, and are a part of a **single context**.



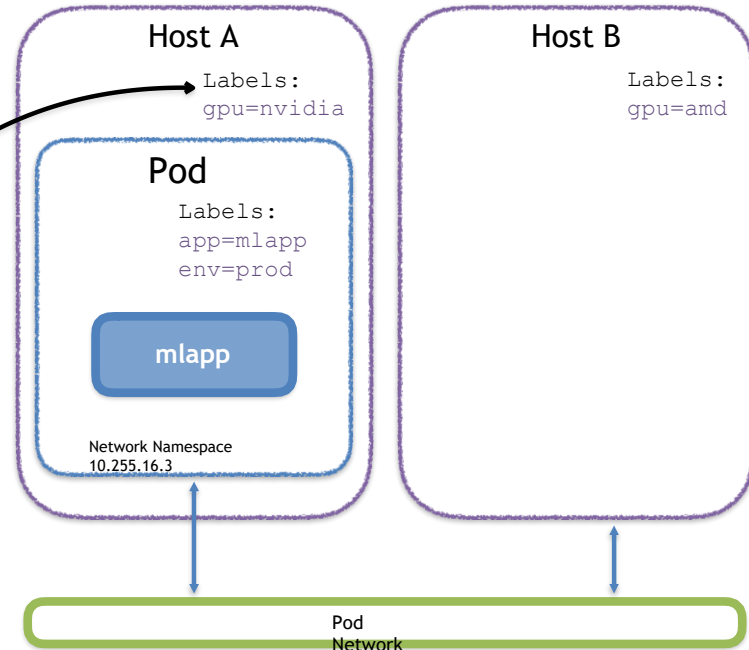
- **key-value** pairs that are used to identify, describe and group together related sets of objects or resources.
- **NOT** characteristic of uniqueness.
- Have a strict syntax with a slightly limited character set*.



<https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/#syntax-and-character-set>

Selectors use **labels** to filter or select objects, and are used throughout Kubernetes.

```
apiVersion: v1
kind: Pod
metadata:
  name: mlapp
  labels:
    app: mlapp
    env: prod
spec:
  nodeSelector:
    - gpu: nvidia
  containers:
    - name: nginx
      image: tensorflow/tensorflow
```



- **Unified method of accessing** the exposed workloads of Pods.
- **Durable resource** (unlike Pods)
 - static cluster-unique IP
 - static namespaced DNS name

There are 4 major service types:

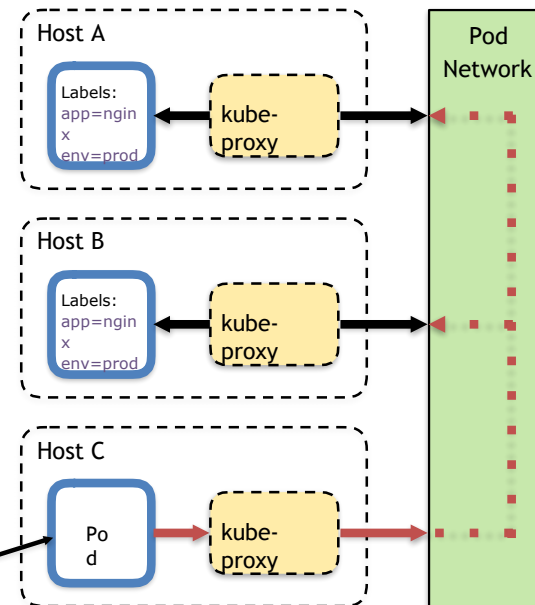
- ClusterIP (default)
- NodePort
- LoadBalancer
- ExternalName

<service name>.<namespace>.svc.cluster.local

- The Pod on host C requests the service.
- Hits host iptables and it load-balances the connection between the endpoints residing on Hosts A, B

```
example-prod
app=nginx,env=prod
ClusterIP
10.96.28.176
<unset>
Name:
Selector:
Type:
IP:
Port:
TargetPort:
```

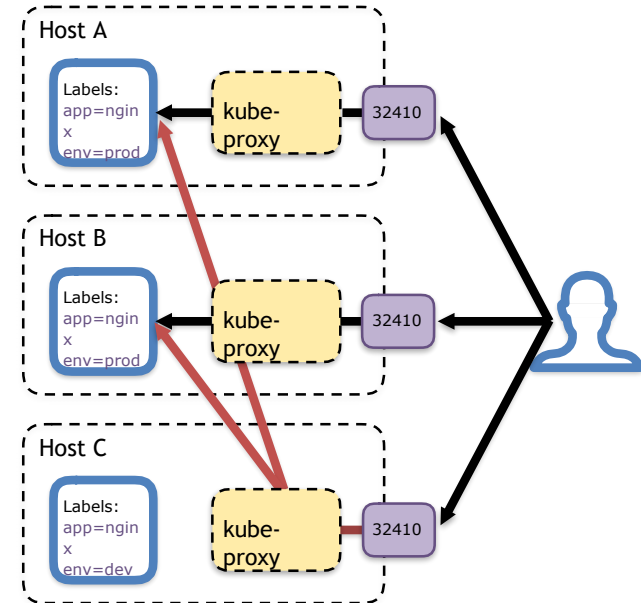
```
Endpoints:
/ # nslookup example-prod.default.svc.cluster.local
Name:      example-prod.default.svc.cluster.local
Address 1: 10.96.28.176 example-prod.default.svc.cluster.local
```



NODE PORT SERVICE

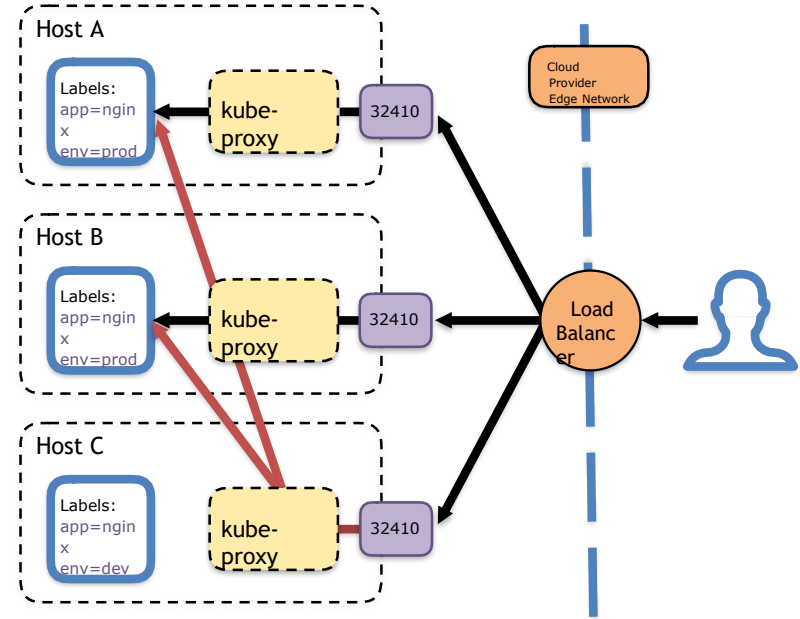
- User can hit any host in cluster on **NodePort** IP and get to service.
- Does introduce extra hop if hitting a host without instance of the pod.

```
Name:          example-prod
Selector:      app=nginx,env=prod
Type:         NodePort
IP:           10.96.28.176
Port:         <unset> 80/TCP
TargetPort:   80/TCP
NodePort:     <unset> 32410/TCP
Endpoints:    10.255.16.3:80,
              10.255.16.4:80
```



- **LoadBalancer** services extend **NodePort**.
- Works in conjunction with an external system to map a cluster external IP to the exposed service.

```
Name:          example-prod
Selector:      app=nginx,env=prod
Type:          LoadBalancer
IP:            10.96.28.176
LoadBalancer  172.17.18.43
Ingress:      <unset> 80/TCP
Port:         80/TCP
TargetPort:   <unset> 32410/TCP
NodePort:     10.255.16.3:80,
Endpoints:    10.255.16.4:80
```



- **ExternalName** is used to reference endpoints **OUTSIDE** the cluster.
- Creates an internal **CNAME** DNS entry that aliases another.

```
apiVersion: v1
kind: Service
metadata:
  name: example-prod
spec:
  type: ExternalName
spec:
  externalName: example.com
```

WORKLOADS

- Workloads within Kubernetes are higher level objects that manage Pods or other higher level objects.
 - In **ALL CASES** a Pod Template is included, and acts the base tier of management.
- ReplicaSet
 - Deployment
 - DaemonSet
 - StatefulSet
 - Job
 - CronJob

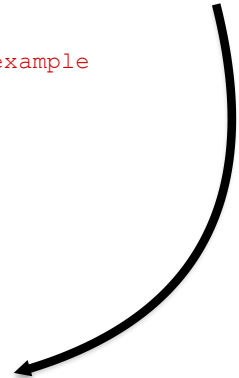
- Workload Controllers manage instances of Pods based off a provided template.
- Pod Templates are Pod specs with limited metadata.
- Controllers use Pod Templates to make actual pods

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
```



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-example
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx
```

```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
    - name: nginx
      image: nginx
```



- **Request:** amount of a resource allowed to be used, with a strong guarantee of availability.

CPU (seconds/second), RAM (bytes)

Scheduler will not over-commit requests

Limit: max amount of a resource that can be used, regardless of guarantees

- Scheduler ignores limits

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-example
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

If we compare a container template with a Class definition in Java, an `initContainer` would be the `constructor` of a class; while a running Pod, would be an instance of that class.

Can be used for everything that should happen **before** the containers within a Pod start running. For example: wait for dependencies, initialize volumes or databases, verify requirements, etc.

- Primary method of managing pod replicas and their lifecycle.
- Includes their scheduling, scaling, and deletion.
- Their job is simple: **Always** ensure the desired number of pods are running.



- **replicas**: The desired number of instances of the Pod.
- **selector**: The label selector for the **ReplicaSet** will manage **ALL** Pod instances that it targets; whether it's desired or not.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-example
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  template:
    <pod template>
```

- Declarative method of managing Pods via ReplicaSets.
- Provide rollback functionality and update control.
- Updates are managed through the **pod-template-hash** label.
- Each iteration creates a unique label that is assigned to both the **ReplicaSet** and subsequent Pods.



- **revisionHistoryLimit**: The number of previous iterations of the Deployment to retain.
- **strategy**: Describes the method of updating the Pods based on the **type**. Valid options are:
 - **Recreate**: All existing Pods are killed before the new ones are created.
 - **RollingUpdate**: Cycles through updating the Pods according to the parameters: **maxSurge** and **maxUnavailable**.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-example
spec:
  replicas: 3
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
      env: prod
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

- Ensure that all nodes matching certain criteria will run an instance of the supplied Pod.
- They **bypass** default scheduling mechanisms.
- Are ideal for cluster wide services such as log forwarding, or health monitoring.
- Revisions are managed via a **controller-revision-hash** label.



- **spec.template.spec.nodeSelector**: The primary selector used to target nodes.
- **Default Host Labels**:
 - `kubernetes.io/hostname`
 - `beta.kubernetes.io/os`
 - `beta.kubernetes.io/arch`
- **Cloud Host Labels**:
 - `failure-domain.beta.kubernetes.io/zone`
 - `failure-domain.beta.kubernetes.io/region`
 - `beta.kubernetes.io/instance-type`

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: ds-example
spec:
  revisionHistoryLimit: 3
  selector:
    matchLabels:
      app: nginx
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
  template:
    spec:
      nodeSelector:
        nodeType: edge
      <pod template>
```

- Tailored to managing Pods that must persist or maintain state.
- Pod identity including **hostname**, **network**, and **storage**

WILL be persisted.

- Assigned a unique ordinal name following the convention of '**<statefulset name>-<ordinal index>**'.
- Naming convention is also used in Pod's network Identity and Volumes.
- Pod lifecycle will be ordered and follow consistent patterns.
- Revisions are managed via a **controller-revision-hash** label.



```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: sts-cassandra
spec:
  replicas: 3
  selector:
    matchLabels:
      app: cassandra
  serviceName: cassandra
  updateStrategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: cassandra
    spec:
      containers:
        - name: cassandra-node
          image: cassandra:3.11.4
          env:
            - name: CASSANDRA_SEEDS
              value: sts-cassandra-0.cassandra
            - name: CASSANDRA_CLUSTER_NAME
              value: my-cluster
          ports:
            - containerPort: 7000
            - containerPort: 7199
          volumeMounts:
            - name: cassandra-data
              mountPath: /cassandra_data
      volumeClaimTemplates:
        - metadata:
            name: cassandra-data
          spec:
            accessModes: [ "ReadWriteOnce" ]
            storageClassName: standard
            resources:
```

The name of the associated headless service; or a service without a ClusterIP

Pods with an ordinal greater than the partition value will be updated one-by-one in reverse order

Template of the persistent volume(s) request to use for each instance of the StatefulSet.

```
<StatefulSet Name>-<ordinal>.<service  
name>.<namespace>.svc.cluster.local
```

```
apiVersion: v1  
kind: Service  
metadata:  
  name: cassandra  
spec:  
  clusterIP: None  
  selector:  
    app: cassandra  
  ports:  
    - name: intra-node  
      port: 80  
    - name: jmx  
      port: 7199  
    - name: cql  
      port: 9042
```

```
$ kubectl get pods
```

	NAME	READY	STATUS	RESTARTS	AGE
	sts-cassandra-0	1/1	Running	0	11m
	sts-cassandra-1	1/1	Running	0	11m
	sts-cassandra-2	1/1	Running	0	11m

```
/ # dig cassandra.default.svc.cluster.local +noall +answer  
;  
;<<>> DiG 9.9.4-RedHat-9.9.4-74.el7_6.1 <<>> cassandra.default.svc.cluster.local +noall +answer  
;; global options: +cmd  
cassandra.default.svc.cluster.local. 5 IN A 172.17.0.5  
cassandra.default.svc.cluster.local. 5 IN A 172.17.0.4  
cassandra.default.svc.cluster.local. 5 IN A 172.17.0.6  
/ # dig sts-cassandra-0.cassandra.default.svc.cluster.local +noall +answer  
;  
;<<>> DiG 9.9.4-RedHat-9.9.4-74.el7_6.1 <<>> sts-cassandra-0.cassandra.default.svc.cluster.local +noall +answer  
/ # dig sts-cassandra-1.cassandra.default.svc.cluster.local +noall +answer  
;  
;<<>> DiG 9.9.4-RedHat-9.9.4-74.el7_6.1 <<>> sts-cassandra-1.cassandra.default.svc.cluster.local +noall +answer  
/ # dig sts-cassandra-2.cassandra.default.svc.cluster.local +noall +answer  
;  
;<<>> DiG 9.9.4-RedHat-9.9.4-74.el7_6.1 <<>> sts-cassandra-2.cassandra.default.svc.cluster.local +noall +answer
```

<Volume Name>.<StatefulSet Name>-<ordinal>

```
volumeClaimTemplates
:- metadata:
  name: data
spec:
  accessModes: [ "ReadWriteOnce"
  storageClassName standard
  : resources:
    requests:
      storage: 1Gi
```

Persistent Volumes associated with a StatefulSet will **NOT** be automatically garbage collected when its associated StatefulSet is deleted. They must manually be removed.

```
$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCE	MODES	STORAGECLASS	AGE
					SS		
data-cassandra-sts-0	Bound	pvc-8baec2a5-8c77-11e9-a6be-0800275ddaf0	100Gi	RWO		standard	17m
data-cassandra-sts-1	Bound	pvc-95463714-8c77-11e9-a6be-0800275ddaf0	100Gi	RWO		standard	17m
data-cassandra-sts-2	Bound	pvc-9807c42a-8c77-11e9-a6be-0800275ddaf0	100Gi	RWO		standard	17m

- Job controller ensures one or more pods are executed and successfully terminate.
- Will continue to try and execute the job until it satisfies the completion and/or parallelism condition.
- Pods are NOT cleaned up until the job itself is deleted.

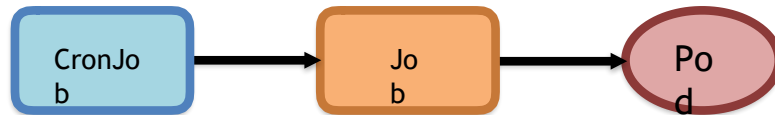


- **backoffLimit**: The number of failures before the job itself is considered failed.
- **completions**: The total number of successful completions desired.
- **parallelism**: How many instances of the pod can be run concurrently.
- **spec.template.spec.restartPolicy**: Jobs only support a restartPolicy of type **Never** or **OnFailure**.

```

apiVersion: batch/v1
kind: Job
metadata:
  name: job-example
spec:
  backoffLimit: 4
  completions: 4
  parallelism: 2
  template:
    spec:
      restartPolicy: Never
      template:
        <pod-template>
  
```

- An extension of the Job Controller, it provides a method of executing jobs on a cron-like schedule.
- CronJobs within Kubernetes use **UTC ONLY**.



- Might be the most powerful feature of K8s.
- When none of the workloads fits the needs of your app, you can create your own controllers with your own specifications (CRD).
- Should be implemented in Go, but there are alternatives.
- A given controller usually requires a ServiceAccount with proper RBAC Role to be able to manage workloads (e.x. create and maintain Pods, Services, etc.)

```
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: opennms-database
spec:
  teamId: OpenNMS
  volume:
    size: 100Gi
  numberOfInstances: 3
  enableMasterLoadBalancer: false
  enableReplicaLoadBalancer: false
  users:
    opennms:
      - superuser
      - createdb
  databases:
    opennms: opennms
  postgresql:
    version: "10"
```

LAB

